PENJELASAN & TUTORIAL DEMO

NAKESA tanpa Supabase, sekarang tersusun MVC

Tampilan, fitur, dan fungsi tetap sama. Yang diganti hanya bagian yang belum dipelajari di kelas (Supabase), sekarang dikerjakan dengan **HTML + JavaScript fetch + PHP** sesuai materi Week 1–6, lalu disusun **MVC** di Week 7.

Sigit Novriyanto · Minggu, 25 Oktober 2026 · PHP 8.5 · `php -S … router.php` · 6 akun demo · diuji di Chrome

Intinya dalam 30 detik

Dulu setiap halaman NAKESA memanggil **Supabase** langsung dari browser untuk login dan data. Supabase belum ada di materi kuliah, jadi sekarang diganti dengan **server PHP sendiri**: browser mengambil dan mengirim data dengan `fetch` ke file di folder `api/`, dan PHP menyimpannya sebagai file JSON, satu folder per akun.

Ke-18 halaman `html/*.html` tetap sama (hanya 2 baris berubah, di `login.html` dan `practice.html`). Di JavaScript, yang diganti hanya baris yang dulu memanggil Supabase. Login dan daftar sekarang memakai **session PHP**, dan ada **6 akun demo** yang langsung berisi data.

Week 7 (MVC): isi `api/*.php` pindah ke 9 class **Controller**, class Week 6 menjadi **Model** (`app/Model/`), halaman HTML tetap menjadi **View**. Profil praktik mengikuti struktur slide PresidentKu: `php/Model`, `View`, `Controller`.

Materi Task 01–03 tiap week di NAKESA

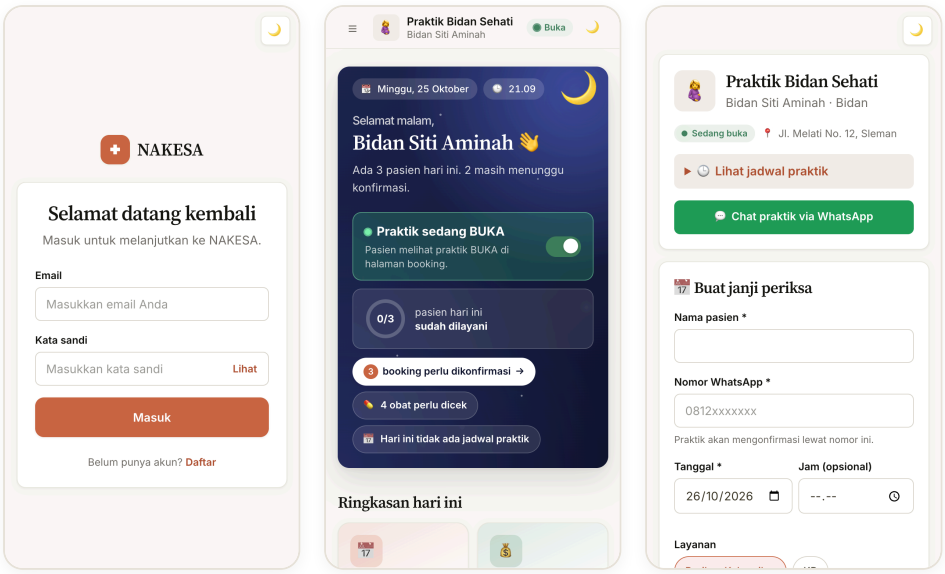
Week	Task 01	Task 02	Task 03
1 · Kickoff	Define your product: pengguna, masalah, solusi, produk, dan fitur utama (bagian 03)		
2 · HTML + CSS	18 halaman HTML semantic (`header`, `nav`, `main`, `section`)	Style & responsive (Tailwind), HP sampai laptop	Refine: menu jadi laci di HP, tanpa horizontal overflow
3 · JS & DOM	Keuangan: select & change judul/subjudul	Keuangan: cari catatan (event `input`) + filter	Keuangan: rincian per kategori
4 · API	`fetch` + JSON: `js/api.js` ke `api/*.php` (dan API libur nasional)	Data API digambar ke DOM (Beranda, Booking, …)	Loading, berhasil, gagal + Coba lagi
5 · PHP	`php/praktik.php` + endpoint `api/*.php`	Proses data: array, conditional, loop (filter booking, saldo, stok)	Konten dinamis: profil praktik, JSON per akun
6 · OOP	10 class server: `Auth`, `User`, `Account`, `Table`, …	Property private + method (`signIn()`, `receive()`, `complete()`)	Object menghasilkan data halaman (inventory, resep, profil publik)
7 · MVC	Struktur `Model/`, `View/`, `Controller/` (aplikasi + profil praktik)	Controller: `PracticeController` (`index()`, `show()`, `process()`) + 9 Controller API	Model → Controller → View: data praktik/booking sampai ke halaman

Isi dokumen

01 Hasilnya
02 Arsitektur: dulu vs sekarang
03 Week 1 — Define the product
04 Week 2 — HTML & responsive
05 Week 3 — JavaScript & DOM
06 Week 4 — Fetch API
07 Week 5 — PHP server-side
08 Week 6 — class PHP (OOP)
09 Week 7 — MVC
10 Login & daftar akun
11 6 akun demo
12 Tutorial demo
13 File yang berubah & pengujian
14 Tanya-jawab & batasan

01 Hasilnya

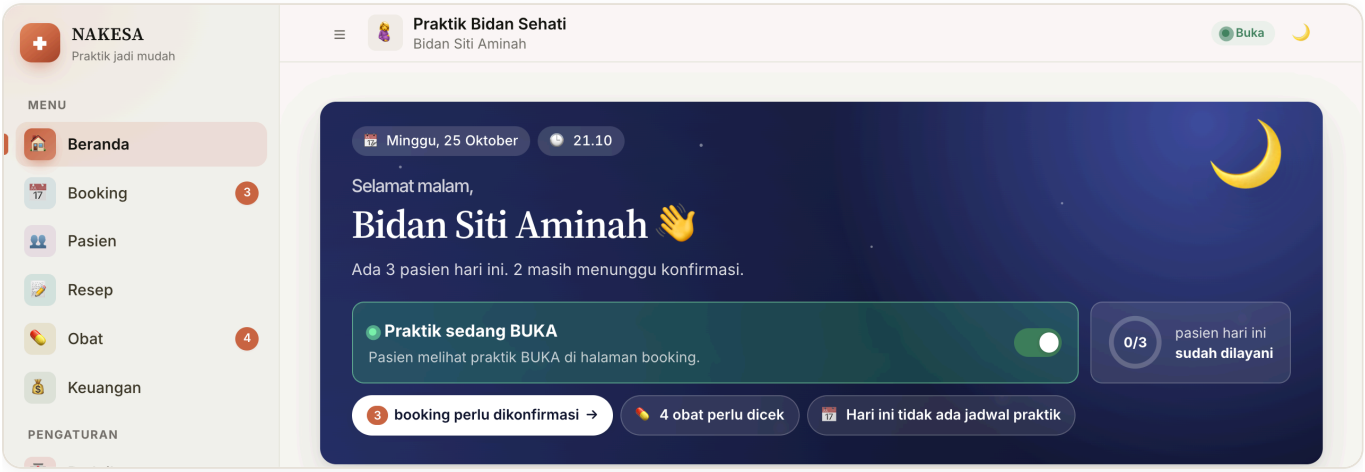
Semua halaman tampil persis seperti NAKESA sebelumnya. Bedanya ada di balik layar: datanya dari server PHP.



Masuk
html/login.html

Beranda
html/dashboard.html

Booking pasien
html/book.html?p=…



Beranda di laptop, akun demo Bidan Siti Aminah. Booking, keuangan 7 hari, obat yang perlu dicek, dan libur nasional dimuat lewat `fetch`.

Yang sama	Yang berubah
Semua halaman HTML, CSS/Tailwind, tata letak, warna, animasi	Login, daftar, keluar: Supabase Auth → `api/auth.php` + session PHP
Semua fitur: booking, pasien, resep, obat & batch, keuangan, jadwal, layanan, link booking + QR, profil praktik	Semua data: `supabase.from(…)` → `fetch('../api/….php')`, disimpan di `data/akun-…/*.json`
Aturan bisnis: stok FEFO, resep mengurangi stok, batas booking 60 hari, maksimal 3 booking/nomor/hari	Aturan yang dulu ditulis di SQL sekarang ditulis di class PHP
Halaman Week 5–7 `php/praktik.php`	Datanya dari server sendiri, tidak lagi lewat cURL ke Supabase; kodenya dipecah ke `php/Model`, `View`, `Controller`
Alamat API (`api/*.php`) dan bentuk JSON-nya	Week 7: isi `api/*.php` pindah ke class Controller; class Week 6 menjadi Model di `app/Model/`
Disembunyikan dulu (butuh layanan luar): masuk dengan Google/Apple, dan lupa kata sandi lewat email. Halamannya tetap disimpan.	

02 Arsitektur: dulu vs sekarang

Dulu (belum sesuai materi)

html/*.html
tampilan
→
js/pages/*.js
DOM & event
→
supabase-js
library dari CDN
→
Supabase
Postgres + Auth + RLS di internet

Sekarang (Week 2 → 7)

html/*.html
tampilan
W2
→
js/pages/*.js
DOM & event
W3
→
js/api.js
`fetch` + JSON
W4
→
api/*.php
18 endpoint
W5
→
app/Controller
9 class W7
→
app/Model
10 class
W6
→
data/akun-.../
file JSON

VIEW biru · CONTROLLER kuning · MODEL hijau (Week 7, bagian 09)

Pengganti setiap bagian Supabase

Dulu (Supabase)	Sekarang	File
Tabel Postgres + `CHECK`	Aturan kolom di array PHP, diperiksa class `Table`	`app/schema.php`, `Table.php`
`supabase.from('patients').insert(row)`	`insertRow('patients', row)` → `POST api/patients.php`	`js/db.js`
`.select().eq().order()`	Endpoint dengan `?view=…`, misalnya `bookings.php?view=today`	`api/*.php`
Row Level Security	Class `Account`: setiap akun hanya membuka foldernya sendiri	`Account.php`
Supabase Auth	Session PHP + `password_hash`	`Auth.php`, `api/auth.php`
View `medicine_inventory`	`Inventory::list()` menghitung stok & status	`Inventory.php`
RPC `receive_stock`, `record_stock_out`, `correct_stock`	`POST api/stock.php?action=receive|out|correct`	`Inventory.php`
RPC `complete_prescription`	`POST api/prescriptions.php?action=complete`	`PrescriptionBook.php`
RPC `get_public_practice`, `create_booking`	`GET / POST api/public.php?slug=…`	`PublicPractice.php`

Contoh: satu baris JavaScript yang berubah

DULU · JS/PAGES/PATIENTS.JS

```
patients = await run(

supabase.from('patients').select('*').order('full_name'));
```

SEKARANG

```
patients = await api('patients.php');
// PHP yang mengurutkan berdasarkan nama
```

Fungsi bantu `insertRow`, `updateRow`, dan `deleteRow` tetap bernama sama, jadi kode halaman yang memakainya tidak perlu diubah.

03 Week 1 — Define the product

Slide "*Mission: Define Your Product*": lima pertanyaan sebelum coding.

Pertanyaan	Jawaban NAKESA
1 · Siapa pengguna kita?	Tenaga kesehatan mandiri (bidan, dokter, dokter gigi, perawat, fisioterapis, psikolog, ahli gizi) yang membuka NAKESA dari HP, serta pasien yang booking lewat link.
2 · Masalah apa yang dihadapi?	Jadwal pasien, catatan pasien, stok obat, dan uang praktik masih dicatat di buku atau chat WhatsApp, sehingga mudah tercecer. Data pasien juga bersifat pribadi.
3 · Solusi apa yang ditawarkan?	Satu aplikasi web: pasien booking sendiri, nakes mengonfirmasi, mencatat pasien, resep, stok, dan keuangan. Setiap nakes masuk dengan akunnya sendiri.
4 · Produk web apa yang dibuat?	NAKESA: 18 halaman HTML + server PHP. Halaman publik: booking (`book.html`) dan profil praktik (`php/praktik.php`).
5 · Fitur utama yang wajib ada?	Daftar/masuk · Beranda · Booking · Pasien · Resep · Obat & stok · Keuangan · Jadwal & status buka · Layanan & harga · Link booking + QR.

Kenapa Supabase diganti? Slide "*Sub-CPMK*" mengurutkan skill: HTML/CSS → JavaScript → API → PHP → OOP → … → CRUD + Auth → Deployment. Supabase melompati urutan itu. Sekarang setiap lapisan NAKESA memakai skill yang sudah dipelajari, dan saat materi database/CRUD/Auth datang, cukup class penyimpanan (`JsonStore`) yang diganti.

Workflow pengembangan (slide "Workflow Pengembangan")

Backlog
Ganti ±40 query + 8 fungsi database Supabase

→

Build
Class PHP, 18 endpoint, lapisan `fetch`

→

Test
curl ke setiap API + Chrome untuk setiap halaman

→

Demo
Bagian 12

→

Improve
Bagian 14

04 Week 2 — HTML semantic & responsive

HTML-nya tetap file `.html` seperti dari awal. Tidak ada halaman yang ditulis ulang.

Task 01 · Struktur HTML semantic

HTML/DASHBOARD.HTML (RINGKAS)

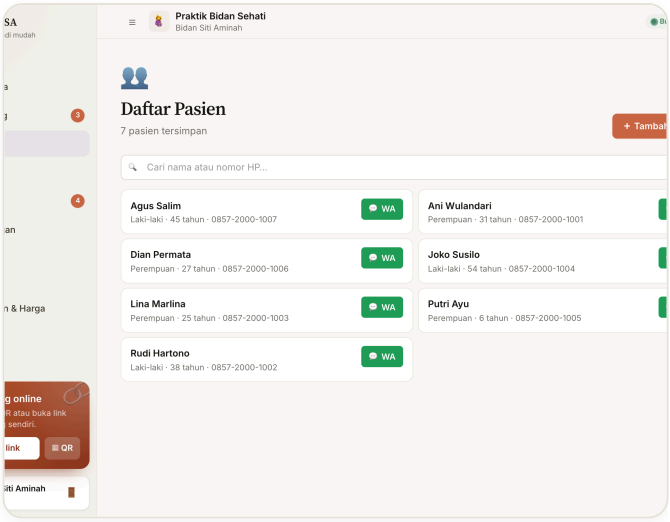
```
<body class="app has-nav">
  <header class="app-header"> … nama praktik, status Buka/Tutup, tombol tema … </header>
  <nav class="app-nav" aria-label="Menu utama"> … Beranda, Booking, Pasien, Resep, Obat, Keuangan … </nav>
  <main id="app-main">
    <section> salam + status praktik </section>
    <section> ringkasan hari ini </section>
    <section> pasien hari ini </section> …
  </main>
</body>
```

Task 02 · Style & responsive



HP · 400px

menu jadi laci (≡), kartu 2×2



Laptop · 1280px

menu tetap di samping

Task 03 · Refine the interface: checklist

Checklist slide	Hasil
Semantic HTML · struktur jelas	✓ header / nav / main / section / footer; setiap input punya `<label>`
CSS diterapkan · layout rapi	✓ Tailwind + komponen bersama di `js/tailwind-setup.js`, tema terang & gelap
Responsive · tanpa horizontal overflow	✓ Diukur otomatis di Chrome (`scrollWidth − clientWidth`) untuk 11 halaman pada lebar 360, 400, dan 1280px: semuanya 0
Bug lama yang ikut diperbaiki	Link booking di **Praktik** membuat halaman melebar 225px, dan kartu di **Layanan & Harga** melebar 15px di HP. Penyebabnya kolom grid yang ikut melebar sepanjang teks; diperbaiki dengan `grid-cols-[minmax(0,1fr)]`. Tampilan lain tidak berubah.
Perubahan HTML lain	`login.html` : `data-requires="reset"` pada link "Lupa kata sandi" supaya bisa disembunyikan

05 Week 3 — JavaScript & DOM (halaman Keuangan)

Ketiga task Week 3 ada di `js/pages/finance.js` dan tetap berjalan; sekarang datanya dari `api/transactions.php`.

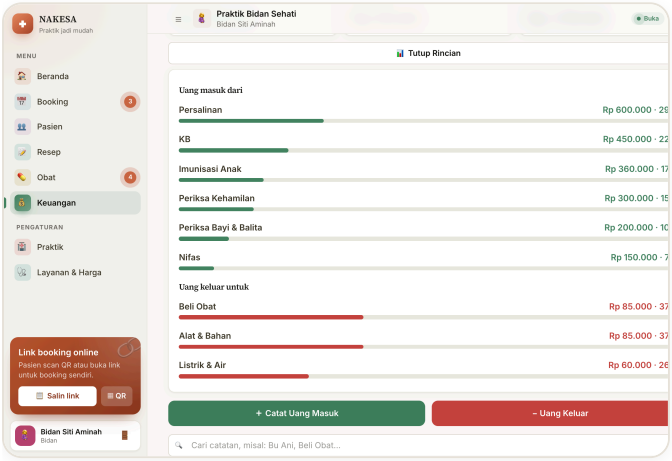
Task 01 · Select & Change

JS/PAGES/FINANCE.JS

```
const title = document.querySelector('h1');                // selector tag
title.textContent = `Keuangan ${practice.name}`;
const subtitle = document.querySelector('#subjudul');      // selector id
subtitle.textContent = 'Catat setiap uang masuk dan keluar supaya untung praktik terlihat jelas.';
const incomeButton = document.querySelector('.btn-success'); // selector class
incomeButton.textContent = '+ Catat Uang Masuk';
```

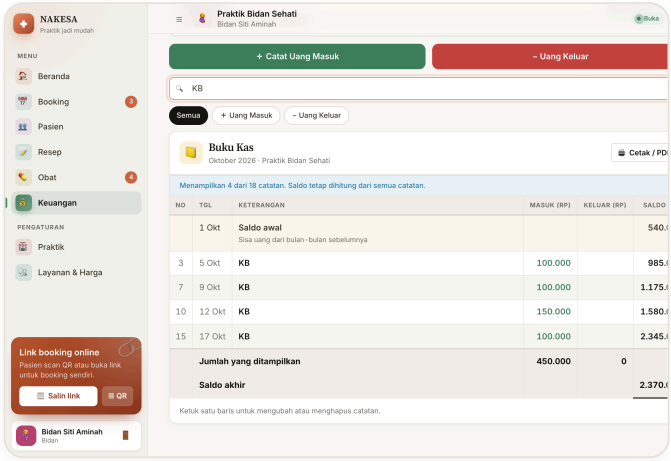
Task 02 · Handle User Event & Task 03 · Build One Complete Interaction

```
search.addEventListener('input', render);                  // T02: daftar tersaring setiap kali mengetik
breakdownButton.addEventListener('click', () => {          // T03: klik → jumlahkan per kategori → tampilkan
  const opening = breakdown.hidden;
  breakdown.hidden = !opening;
  breakdownButton.textContent = opening ? '📊 Tutup Rincian' : '📊 Lihat Rincian per Kategori';
  renderBreakdown();
});
```



Task 03

rincian uang masuk/keluar per kategori



Task 02

mengetik "KB" → hanya catatan KB

Interaksi DOM lain yang tetap sama: tombol *Lihat* kata sandi, konfirmasi booking, pop-up pasien, pemilih obat di resep, jadwal per hari, QR link booking.

06 Week 4 — Fetch API

Dulu Week 4 hanya dipakai untuk kartu libur nasional. Sekarang **seluruh** data NAKESA lewat REST API buatan sendiri.

Task 01 · Connect to API: satu fungsi untuk semua request

JS/API.JS

```
export async function api(path, { method = 'GET', params, body } = {}) {
  const url = new URL(path, API_BASE);                          // …/api/patients.php
  for (const [key, value] of Object.entries(params ?? {})) url.searchParams.set(key, value);
  const response = await fetch(url, { method, body: body === undefined ? undefined : JSON.stringify(body),
    headers: …, credentials: 'same-origin' });                  // ikut mengirim cookie login
  const data = await response.json().catch(() => null);
  if (!response.ok) throw Object.assign(new Error(data?.message), { status: response.status, userMessage:
data?.message });
  return data;
}
```

Method HTTP	Contoh di NAKESA	Arti
`GET`	`api/bookings.php?view=today&date=2026-10-25`	Ambil booking hari ini
`POST`	`api/patients.php` + body JSON	Tambah pasien
`PATCH`	`api/bookings.php?id=…` body `{"status":"dikonfirmasi"}`	Ubah sebagian kolom
`PUT`	`api/practice_hours.php` body daftar sesi	Ganti seluruh jadwal
`DELETE`	`api/transactions.php?id=…`	Hapus catatan keuangan

Task 02 · Display API Data · Task 03 · Handle API State

JS/PAGES/DASHBOARD.JS · LOAD()

```
const [bookings, pending, money, patients, medicines, practiceHours] = await Promise.all([
  api('bookings.php', { params: { view: 'today', date: today } }),     // 6 request sekaligus
  api('bookings.php', { params: { view: 'pending_count', date: today } }),
  api('transactions.php', { params: { from: todayISO(-6) } }), …
]);
renderStats(…); renderToday(); …                                       // T02: JSON → DOM
} catch (error) { toast(appError(error), 'error'); }                     // T03: pesan dari server
                                                                         // atau "Koneksi internet
bermasalah"
```

JAWABAN API/PRACTICE_HOURS.PHP (JSON)

```
[{"id":"669af086-…","owner_id":"8f1c2a30-
0001-…","day_of_week":1,"opens_at":"08:00:00","closes_at":"12:00:00", …}, …]
```

BELUM LOGIN → 401

```
{"message":"Sesi Anda sudah berakhir. Silakan masuk lagi.","code":"not_signed_in"}
```

State per halaman: **loading** (`#loader` berputar sampai data datang) → **berhasil** (halaman tampil) → **gagal** (toast merah dengan pesan dari server). Kartu libur nasional tetap memakai state *Memuat… → Data berhasil dimuat → Gagal dimuat* + ↻ *Coba Lagi*.

07 Week 5 — PHP server-side

Pola INPUT → PROCESS → OUTPUT dari Week 5 dipakai di setiap endpoint: baca `?view=`/body, olah array dengan conditional dan loop, lalu kirim JSON. (Sejak Week 7, bagian PROCESS berada di class Controller; lihat bagian 09.)

Task 01–02 · Endpoint: array, conditional, loop

API/BOOKINGS.PHP (POTONGAN)

```
$controller = new BookingController(account());        // INPUT: akun yang login (401 kalau belum)
$date = query('date') ?: todayISO();

if (method() === 'GET') {
    $view = query('view');
    if ($view === 'today') {                              // PROCESS: conditional
        respond($controller->today($date));
    }
    if ($view === 'pending_count') {
        respond($controller->pendingCount($date));         // ['count' => count(booking 'baru')]
    }
    …
}
if (method() === 'PATCH') { respond($controller->update(requireId(), requestBody())); }   // OUTPUT: JSON
```

APP/CONTROLLER/TRANSACTIONCONTROLLER.PHP · BALANCE() — SALDO SEBELUM SUATU TANGGAL

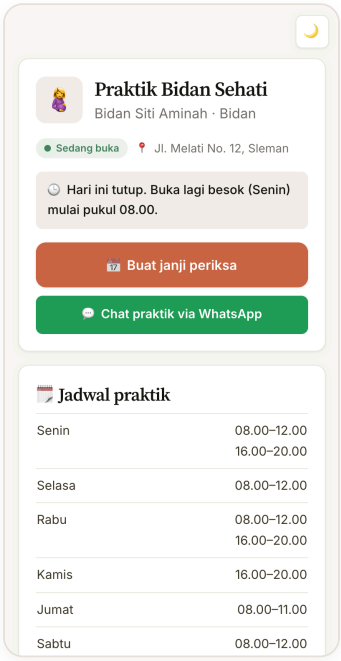
```
$balance = 0;
foreach ($this->transactions->all() as $t) {           // loop
    if ($t['occurred_on'] < $before) {
        $balance += $t['kind'] === 'masuk' ? $t['amount'] : -$t['amount'];
    }
}
return ['balance' => $balance];
```

Task 03 · Dynamic content: `php/praktik.php`

Halaman tugas Week 5–6 dikembalikan dan tetap dibuat PHP di server. Sumber datanya diganti: dulu cURL ke Supabase, sekarang `PublicPractice`, yang juga dipakai halaman booking.

```
$model = new PracticeModel(
    new PublicPractice($userRepo),
$samplePractice);
$practice = $model->findBySlug($slug);
```

Contoh: `/php/praktik.php?p=praktik-bidan-sehati`. Di Week 7, kode ini dibagi ke Model, Controller, dan View (bagian 09).



php/praktik.php
jadwal hari ini dari data Bidan Siti

08 Week 6 — Class PHP (OOP)

Logika server dibagi ke 10 class di `app/Model/` (sebelum Week 7 bernama `app/classes/`). Setiap property `private`; dari luar hanya lewat method.

Auth
masuk, keluar, siapa yang login

```
- $users : UserRepository
- $user : ?User
+ signIn($email, $password) : User
+ login(User), logout()
+ currentUser() : ?User
+ requireUser() : User
```

User · UserRepository
akun di data/users.json

```
- $id, $email, $fullName
- $profession, $passwordHash
+ verifyPassword($password)
+ toProfile() : array
+ register(…) : User
+ findByEmail(), updateProfile()
```

Account
data milik akun yang login

```
- $user : User
- $practice : ?array
+ table($name) : Table
+ practice(), createPractice()
+ updatePractice()
+ transaction(callable)
```

Table · JsonStore
satu tabel = satu file JSON

```
- $store, $columns, $fixed
+ all(), find($id), where($filter)
+ insert(), insertMany()
+ update(), delete()
+ validate($input) — aturan schema.php
```

Inventory
stok obat (dulu view + 4 fungsi SQL)

```
- $account : Account
+ list(), find(), attentionCount()
+ receive(), stockOut(), correct()
+ dispense($items) — FEFO
+ addExamples(), deleteMedicine()
```

PrescriptionBook · PublicPractice
resep · profil & booking publik

```
- $account / - $users
+ list(), create(), replaceItems()
+ complete($id)
+ find($slug) : ?array
+ book($slug, $input) : string
```

Ditambah class Week 6 untuk halaman profil praktik (`Practice`, `Profession`, `Schedule`, `PracticeRepository`). Di Week 7 class itu pindah ke `php/Model/`, dan `PracticeRepository` digabung menjadi `PracticeModel`.

Contoh method: stok keluar mengikuti aturan yang dulu ada di SQL

APP/MODEL/INVENTORY.PHP · APPLYMOVEMENTS()

```
foreach ($moves as [$batchId, $type, $quantity, $note, $itemId]) {
    $batch = &$batches[$index[$batchId]];
    if ($batch['quantity'] + $quantity < 0) {
        throw new AppError("Stok tidak cukup. Sisa stok di batch ini: {$batch['quantity']}.");
    }
    $batch['quantity'] += $quantity;                    // stok batch berubah…
    $rows[] = ['movement_type' => $type, 'quantity' => $quantity,
               'stock_after' => $batch['quantity'], …];   // …dan riwayatnya dicatat
}
```

Task 03 · Object menghasilkan data halaman

Halaman	Object & method	Hasil
Database Obat	`(new Inventory($account))->list()`	Stok, batch, "Menipis", "Akan kedaluwarsa", badge 🔴 di menu
Resep	`$book->complete($id)` → `Inventory::dispense()`	Status "Selesai", stok berkurang dari batch yang paling cepat kedaluwarsa
Booking pasien	`$public->find($slug)`, `$public->book(…)`	Profil + jadwal praktik; booking baru berstatus "Perlu konfirmasi"

09 Week 7 — MVC: Model, View, Controller

Slide "*From OOP to MVC*": class dari Week 6 sekarang dikelompokkan menurut tanggung jawabnya. Tampilan dan fitur tidak berubah; yang berubah susunan kodenya.

Task 01 · Design the MVC structure

Komponen	Tanggung jawab (slide)	Seluruh aplikasi NAKESA	Halaman profil praktik (`php/`)
MODEL	Mengelola data dan representasi data	`app/Model/` : 10 class (`Auth` , `Account` , `Table` , `Inventory` , `PrescriptionBook` , …) + file JSON di `data/`	`php/Model/` : `PracticeModel` (mengambil data) + object `Practice` , `Profession` , `Schedule`
VIEW	Menampilkan interface kepada pengguna	`html/*.html` + `js/pages/*.js` : 18 halaman yang menggambar data JSON ke DOM	`php/View/practice-page.php` : template HTML dengan `<?= … ?>` , seperti contoh slide
CONTROLLER	Menerima request dan mengatur proses aplikasi	`api/*.php` (pintu masuk, 18 file) → 9 class di `app/Controller/`	`php/praktik.php` (front controller) → `php/Controller/PracticeController.php`

SELURUH APLIKASI

```
NAKESA/
├ html/, js/pages/      // VIEW
├ api/*.php             // CONTROLLER: pintu masuk
└ app/
   ├ Controller/        // CONTROLLER: 9 class
   │  ├ AuthController.php
   │  ├ PracticeSettingsController.php
   │  ├ BookingController.php
   │  ├ PatientController.php
   │  ├ TransactionController.php
   │  ├ MedicineController.php
   │  ├ StockController.php
   │  ├ PrescriptionController.php
   │  └ PublicBookingController.php
   └ Model/             // MODEL: 10 class
      ├ Auth, User, UserRepository
      ├ Account, Table, JsonStore, AppError
      └ Inventory, PrescriptionBook,
        PublicPractice
```

PROFIL PRAKTIK · SEPERTI SLIDE PRESIDENTKU

```
php/
├ praktik.php        // front controller
├ Model/
│  ├ PracticeModel.php   ≈ President.php
│  ├ Practice.php
│  ├ Profession.php
│  └ Schedule.php
├ View/
│  └ practice-page.php   ≈ president-page.php
└ Controller/
   └ PracticeController.php
      ≈ PresidentController.php
      index() · show() · process()

// Contoh data: Praktik Bidan Sehati
//   nama, nakes, profesi  ≈ name
//   jadwal mingguan       ≈ period
//   daftar layanan        ≈ programs
```

Kenapa View aplikasi tetap file HTML? Sejak awal tampilan NAKESA berupa file `.html` (Week 2) yang mengambil data dengan `fetch` (Week 4). Jadi di aplikasi, Controller menjawab **data JSON**, lalu View (HTML + JS) yang menggambarnya. Pola View berupa template PHP, persis seperti slide, dipakai di halaman profil praktik, satu-satunya halaman yang HTML-nya dibuat server.

Checklist Task 01	Hasil
Model ditentukan	✓ `app/Model/` dan `php/Model/` : hanya data + aturannya, tanpa HTML
View ditentukan	✓ `html/*.html` dan `php/View/practice-page.php` : hanya tampilan, tidak membaca file data sendiri
Controller ditentukan	✓ `api/*.php` + `app/Controller/` dan `php/Controller/`
Tanggung jawab masing-masing jelas	✓ setiap file diberi komentar `// MODEL …` , `// VIEW …` , `// CONTROLLER …` di baris pertama

09 Week 7 — Task 02 · Build the Controller

Controller menerima request, menjalankan proses, memanggil Model, lalu mengirim hasilnya ke View. Controller tidak berisi HTML dan tidak membaca file data sendiri.

Profil praktik: `PracticeController` dengan `index()`, `show()`, `process()` seperti slide

PHP/CONTROLLER/PRACTICECONTROLLER.PHP (RINGKAS)

```php
class PracticeController
{
    private PracticeModel $model;                // Controller memegang Model

    public function index(array $query): void    // REQUEST: ?p=<kode link> atau tanpa ?p=
    {
        $slug = is_string($query['p'] ?? null) ? trim($query['p']) : '';
        $this->show($slug === '' ? null : $slug);
    }

    public function show(?string $slug): void    // PROCESS: minta object Practice ke Model
    {
        $practice = $slug === null ? $this->model->sample() : $this->model->findBySlug($slug);
        if ($practice === null) { $this->showError(404, …); return; }   // (gagal baca data → 503)
        $this->render($this->process($practice, $slug === null));
    }

    public function process(?Practice $practice, bool $isSample, ?array $error = null): array
    {   return ['practice' => $practice, 'schedule' => …, 'isSample' => $isSample, …];  }

    private function render(array $data): void   // ke VIEW: ['practice' => …] jadi $practice
    {   extract($data, EXTR_SKIP);   require __DIR__ . '/../View/practice-page.php';  }
}
```

Setiap kemungkinan request ditangani Controller

Request	Yang dikerjakan Controller	Yang tampil (View)
`php/praktik.php`	`show(null)` → `$model->sample()`	Contoh profil + catatan "Ini contoh tampilan"
`?p=praktik-bidan-sehati`	`show($slug)` → `$model->findBySlug()` → object `Practice`	Profil asli: status buka, jadwal, layanan, tombol booking & WhatsApp
`?p=tidak-ada`, `?p=..`	Model menjawab `null` → kode HTTP 404	🔍 "Profil praktik tidak ditemukan"
File data rusak / tidak terbaca	`catch (Throwable)` → dicatat di log server, kode HTTP 503	⚠️ "Data praktik tidak bisa dimuat" + tombol Coba lagi

Method di slide	Di `PracticeController`
`index()`	Menerima request: membaca `?p=` dari `$_GET` (dikirim `praktik.php`)
`show()`	Memanggil Model dan memilih hasil: profil, 404, atau 503
`process()`	Menyusun array data untuk View (judul tab, jadwal, layanan, hari ini). Tidak berisi HTML
render() (private)	Mengirim data ke View: `extract()` lalu `require` `View/practice-page.php`

09 Week 7 — Task 02 · Controller di seluruh aplikasi

Setiap file `api/*.php` sekarang hanya pintu masuk: membaca method HTTP, `?view=` / `?action=`, dan body, lalu memanggil method Controller. Alamat API untuk JavaScript tidak berubah.

Contoh: booking hari ini

API/BOOKINGS.PHP · PINTU MASUK REQUEST

```
$controller = new BookingController(account());
$date = query('date') ?: todayISO();

if (method() === 'GET') {
  $view = query('view');
  if ($view === 'today') {
    respond($controller->today($date));
  }
  …
}
if (method() === 'PATCH') {
  respond($controller->update(
    requireId(), requestBody()));
}
methodNotAllowed();          // 405
```

APP/CONTROLLER/BOOKINGCONTROLLER.PHP

```
class BookingController
{
  private Table $bookings;     // MODEL

  public function __construct(Account $account)
  {
    $this->bookings =
      $account->table('bookings');
  }

  public function today(string $date): array
  {
    return sortRows($this->bookings->where(
      fn ($b) => $b['booking_date'] === $date
        && $b['status'] !== 'batal'),
      self::BY_TIME);
  }
  … pendingCount(), upcoming(), history(), …
}
```

Controller	Dipanggil dari `api/…`	Method utama	Model
`AuthController`	auth, profile	status, login, signup, logout, profile	Auth, UserRepository
`PracticeSettingsController`	practices, practice_hours, practice_services	practice, updatePractice, hours, replaceHours, services	Account, Table
`BookingController`	bookings	index, today, pendingCount, upcoming, history, update	Table
`PatientController`	patients	index, counts, store, update, destroy	Account, Table
`TransactionController`	transactions	index, balance, store, update, destroy	Table
`MedicineController`	medicine_inventory, medicines, medicine_categories, suppliers	index, show, store, destroy, addExamples, attentionCount	Inventory, Table
`StockController`	stock, stock_movements, medicine_batches	receive, stockOut, correct, history, batches	Inventory
`PrescriptionController`	prescriptions, prescription_items	index, store, complete, replaceItems	PrescriptionBook
`PublicBookingController`	public (pasien, tanpa login)	profile, book	PublicPractice

Checklist Task 02: ✓ Controller berjalan · ✓ request diterima (`?p=` , `?view=` , `?action=` , body JSON) · ✓ method dapat dijalankan · ✓ Model dapat dipanggil · ✓ View dapat ditampilkan

09 Week 7 — Task 03 · Connect Model, Controller & View

Alur lengkap dari request sampai tampilan, seperti slide "*Alur Model → Controller → View*".

Profil praktik (HTML dibuat di server)

USER buka `praktik.php?p=…` → **CONTROLLER** `index()` → `show()` → **MODEL** `findBySlug()` → `Practice` → **CONTROLLER** `process()` menyusun data → **VIEW** `practice-page` → HTML

Halaman aplikasi (View HTML + fetch)

VIEW `bookings.js` fetch `?view=today` → **CONTROLLER** `bookings.php` → `today()` → **MODEL** `Table bookings` → array → **CONTROLLER** `respond()` → JSON → **VIEW** menggambar `bookings.html`

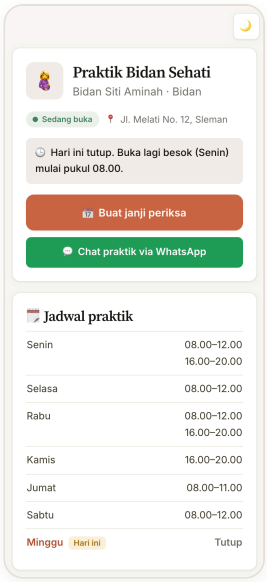
PHP/VIEW/PRACTICE-PAGE.PHP (POTONGAN)

```
<h1><?= e($practice->getName()) ?>
</h1>
<p><?= e($practice->getSubtitle()) ?>
</p>
<?php foreach ($services as $i =>
$service): ?>
  <li><?= $i + 1 ?> · <?= e($service)
?></li>
<?php endforeach; ?>
```

Checklist Task 03	
Data berasal dari Model	✓ `PracticeModel`, `Table`
Controller memproses data	✓ `process()`, `today()`
Data dikirim ke View	✓ `render()`, JSON
View menampilkan data	✓ 3 gambar di kanan
Tidak ada error	✓ bagian 13



Tanpa ?p= `sample()`



?p=praktik-bidan-sehati `findBySlug()`



?p=tidak-ada
Controller: 404

Isi data View hanya dari Controller: `practice-page.php` tidak membuka file JSON, dan Controller tidak menulis HTML. Warna kartu layanan (urusan tampilan) juga dipindah dari Controller ke View.

09 Week 7 — Demo & review

Slide "*Demo: Show Your MVC Flow*" dan "*Review: Is the Application Organized?*"

Jawaban 5 pertanyaan demo

Pertanyaan	Jawaban di NAKESA
1 · Apa fungsi Controller?	Menerima request (`?p=` , `?view=` , `?action=` , body), memilih method yang sesuai, memanggil Model, menangani kegagalan (404, 503, pesan error JSON), lalu menyerahkan data ke View.
2 · Apa data yang dikelola Model?	Akun & login, praktik, jadwal, layanan, pasien, booking, keuangan, obat, batch, riwayat stok, dan resep, beserta aturannya (isian wajib, stok FEFO, batas booking). Di profil praktik: nama, nakes, profesi, jadwal, layanan.
3 · Apa yang ditampilkan View?	Profil: identitas praktik, status buka, jadwal dengan tanda "Hari ini", layanan, tombol booking & WhatsApp, atau pesan "tidak ditemukan". Aplikasi: 18 halaman HTML (Beranda, Booking, Pasien, Resep, Obat, Keuangan, …).
4 · Bagaimana Controller berkomunikasi dengan Model?	Lewat object: Model diberikan saat Controller dibuat (`new PracticeController($model)` , `new BookingController(account())`), lalu Controller memanggil method-nya (`$this->model->findBySlug()` , `$this->bookings->where()`).
5 · Bagaimana data sampai ke View?	Profil: `process()` menyusun array → `render()` menjadikannya variabel → `practice-page.php` menulis HTML. Aplikasi: method Controller mengembalikan array → `respond()` mengirim JSON → `fetch` di `js/pages/*.js` menggambarnya.

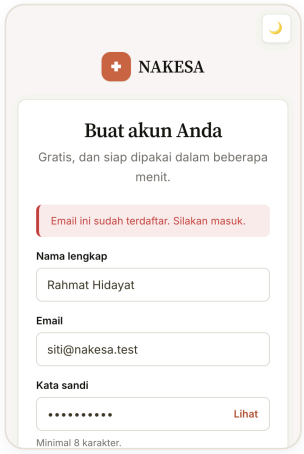
Review checklist

Checklist slide	Bukti di NAKESA
Model memiliki tanggung jawab yang jelas	✓ Hanya Model yang membaca/menulis data dan memeriksa aturannya (`Table::validate()` , `Inventory::dispense()`)
View fokus pada tampilan	✓ `practice-page.php` dan `html/*.html` hanya menampilkan data yang diterima
Controller menangani alur aplikasi	✓ Memilih method dari request, menangani 404/405/503, memanggil Model
Data tidak bercampur dengan tampilan	✓ Tidak ada HTML di Controller/Model; tidak ada akses data di View
Alur MVC dapat dijelaskan	✓ Dua alur di Task 03 + naskah demo langkah 10–11
Halaman berjalan dengan benar · tidak ada error	✓ 100 request API dan profil praktik sama persis dengan sebelum MVC; Chrome tanpa error JS (bagian 13)

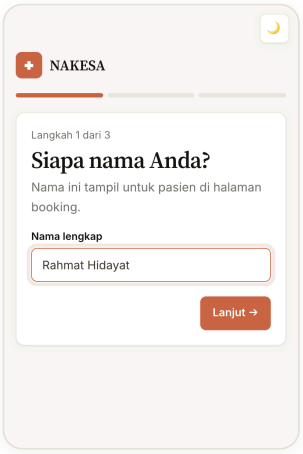
Review: bagian mana yang jadi lebih mudah dikelola? Menambah tampilan data baru cukup satu method di Controller (misalnya `BookingController::upcoming()`) tanpa menyentuh HTML. Desain profil praktik bisa diubah di `practice-page.php` tanpa menyentuh pengambilan data. Saat materi database datang, cukup Model yang diganti; Controller dan View tetap.

10 Login & daftar akun

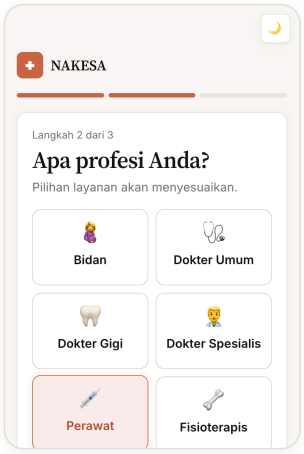
Alurnya sama dengan NAKESA sebelumnya: **Daftar → isi data praktik (3 langkah) → Beranda**. Bedanya, sekarang dikerjakan PHP.



Daftar
email sudah dipakai



Langkah 1
nama



Langkah 2
profesi



Beranda
akun baru, masih kosong

1 · login.html
form email + kata sandi

→

2 · js/auth.js
`fetch POST api/auth.php?action=login`

→

3 · Auth::signIn()
`password_verify()` dengan hash di users.json

→

4 · Session
`$_SESSION['user_id']` ; browser hanya membawa cookie acak

→

5 · dashboard.html
setiap `api/*.php` membaca akun dari session

Ancaman	Penangkal
File akun bocor	Kata sandi disimpan sebagai hash bcrypt (`password_hash`), tidak pernah dikirim ke browser
Membuka data akun lain	Folder data dipilih dari session, bukan dari URL. Mencoba id milik akun lain → 404 (sudah diuji)
File JSON diunduh langsung	`router.php` (php -S) dan `.htaccess` (XAMPP) menolak `/data/` dan `/app/` → 404; `data/` tidak di-commit
Session dicuri / ditebak	`session_regenerate_id()` setiap login; cookie `httponly` + `SameSite=Lax`
Browser mengirim kolom yang tidak boleh	`Table::validate()` hanya menerima kolom di `schema.php` ; `id` , `owner_id` , `created_at` diisi server
Isian salah / terlalu panjang	Aturan yang sama dengan `CHECK` di database lama, dengan pesan berbahasa Indonesia
Dua penyimpanan bersamaan	Kunci file (`flock`) + tulis ke file sementara lalu `rename` , jadi file tidak pernah setengah tertulis

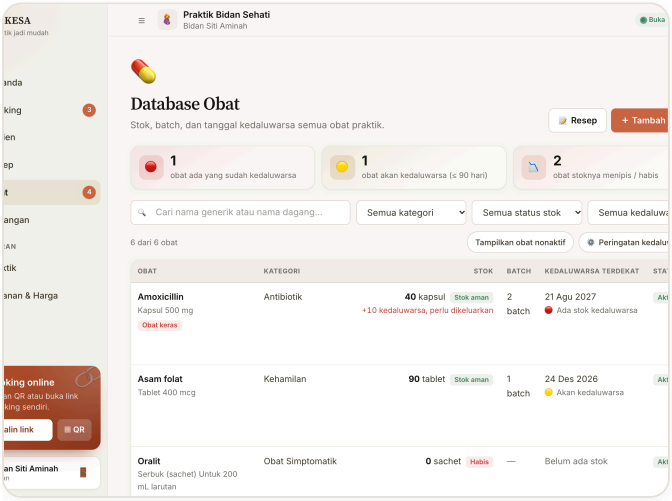
Di luar materi Week 1–7 Session, `password_hash` , dan penguncian file belum dibahas di kelas. Semuanya fungsi bawaan PHP (tanpa library atau layanan luar) dan sudah ditandai di komentar kode.

11 6 akun demo

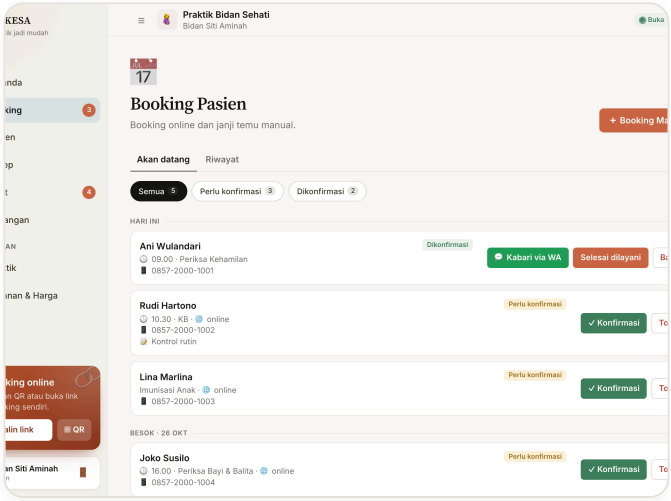
Kata sandi semua akun: `nakesa123`. Data awal di `app/data/contoh/` disalin otomatis ke `data/` saat aplikasi pertama kali dibuka.

Email	Nama tampil	Profesi	Praktik · kode link	Isi data
`siti@nakesa.test`	Bidan Siti Aminah	🤰 Bidan	Praktik Bidan Sehati · `praktik-bidan-sehati`	7 pasien, 9 booking, 6 obat, 2 resep
`andi@nakesa.test`	dr. Andi Pratama	🩺 Dokter Umum	Klinik Sehat Pratama · `klinik-sehat-pratama`	6 pasien, 8 booking, 6 obat, 2 resep
`rina@nakesa.test`	drg. Rina Kartika	🦷 Dokter Gigi	Klinik Gigi Senyum Rina · `klinik-gigi-senyum-rina` (tutup)	5 pasien, 7 booking
`budi@nakesa.test`	Ns. Budi Santoso	💉 Perawat	Rawat Luka & Home Care Budi · `rawat-luka-home-care-budi`	4 pasien, 9 booking, 6 obat, 2 resep
`dewi@nakesa.test`	Dewi Lestari	🦴 Fisioterapis	Fisioterapi Dewi · `fisioterapi-dewi`	7 pasien, 8 booking
`maya@nakesa.test`	Maya Putri	🧠 Psikolog	Ruang Cerita Maya · `ruang-cerita-maya` (tutup)	6 pasien, 7 booking

Setiap akun juga punya jadwal praktik, layanan + harga, dan catatan keuangan 40 hari terakhir. Akun yang memberi obat (bidan, dokter umum, perawat) sengaja dibuat punya obat dengan stok **menipis**, **habis**, **akan kedaluwarsa**, dan batch yang **sudah kedaluwarsa**, supaya semua status dan badge terlihat saat demo. Tanggal di data contoh ditulis relatif (`"hari"`, `"hari+1"`, `"hari-3 09:15"`), jadi booking "hari ini" selalu ada kapan pun demo dijalankan.



Database Obat · Bidan Siti
status stok & kedaluwarsa



Booking · Bidan Siti
hari ini, besok, dan seterusnya

12 Tutorial demo

Persiapan (sekali, sebelum kelas)

1. Buka Terminal di folder proyek NAKESA, lalu jalankan:

   ```
   php -S localhost:8000 router.php
   ```

   XAMPP: salin folder ke `htdocs/` (`.htaccess` sudah ikut), lalu buka `http://localhost/NAKESA/`.

2. Ingin mulai dari data awal? Hapus folder `data/`.

3. Buka `http://localhost:8000/`.

Naskah demo ± 8 menit

#	Lakukan	Katakan
1	Buka `html/dashboard.html` tanpa login	"Belum login, jadi saya dipindah ke halaman Masuk. API menjawab 401." **W6 Auth**
2	Masuk: `siti@nakesa.test` / kata sandi salah	"Pesan ini datang dari PHP, lewat fetch." **W4 T3**
3	Masuk dengan `nakesa123`	"Tampilannya sama persis dengan NAKESA sebelumnya, tapi tanpa Supabase."
4	DevTools → Network → muat ulang Beranda	"Ada 6 request ke `api/*.php`, semuanya menjawab JSON." **W4 T1–T2**
5	Tekan ✓ **Konfirmasi** pada booking hari ini	"Satu PATCH ke `bookings.php`; angka di menu ikut turun."
6	Keuangan → ketik di kotak cari → **Lihat Rincian**	"Task Week 3: event input dan interaksi lengkap." **W3 T1–T3**
7	Resep → Detail draf → **Selesaikan & kurangi stok**	"Class PrescriptionBook memakai Inventory: stok berkurang dari batch yang paling cepat kedaluwarsa." **W6**
8	Obat → Amoxicillin → **Keluarkan (kedaluwarsa)**	"Riwayat stok tercatat dan tidak bisa dihapus."
9	Buka link booking di tab penyamaran, isi, kirim	"Pasien tanpa login; `api/public.php` hanya memberi data untuk pasien."
10	Buka `/php/praktik.php?p=praktik-bidan-sehati`, lalu `?p=tidak-ada`	"Request masuk ke PracticeController, data diambil PracticeModel, HTML ditulis View. Kode salah → Controller menampilkan 404." **W5–W7**
11	Buka folder `php/` dan `app/` di VS Code: `Model/`, `View/`, `Controller/`	"Ini struktur MVC dari Task 01. Saya telusuri satu request: `api/bookings.php` → `BookingController::today()` → Model `Table` → JSON → `bookings.js`." **W7 T1–T3**
12	Keluar → **Daftar** akun baru → isi 3 langkah	"Akun baru langsung punya folder data sendiri, masih kosong."
13	Buka `/data/users.json`	"404: file akun tidak bisa diunduh dari browser."

13 File yang berubah & pengujian

File	Isi
`api/*.php` *baru, 18 file*	Satu file per jenis data (auth, profile, practices, patients, bookings, transactions, medicines, stock, prescriptions, public, …)
`app/Model/*.php` *baru, 10 class*	AppError, JsonStore, Table, User, UserRepository, Auth, Account, Inventory, PrescriptionBook, PublicPractice (Week 6, folder `app/classes/` diganti nama di Week 7)
`app/Controller/*.php` *baru Week 7, 9 class*	Isi 18 file `api/*.php` dipindah ke sini (518 baris); setiap `api/*.php` tinggal meneruskan request ke method Controller
`app/config.php`, `schema.php`, `helpers.php`, `bootstrap.php`, `api.php`	Pengaturan, aturan kolom setiap tabel, fungsi bantu, pemuat class, dasar API (JSON + error)
`app/data/contoh/` *baru*	`users.json` (6 akun) + satu folder data per akun
`js/api.js` *baru*; `js/db.js`, `auth.js`, `profile.js`, `roles.js`	Lapisan `fetch`; nama fungsi tetap sama dengan versi Supabase
`js/pages/*.js`, `shell.js`, …	Hanya baris yang memanggil Supabase diganti (23 file JS, +158 / –285 baris)
`js/config.js`, `errors.js`, `ui.js`, `html/login.html`	Google/Apple dan "lupa kata sandi" disembunyikan; pesan error dari server
`html/practice.html`, `js/tailwind-setup.js` (`.list`)	Perbaikan horizontal overflow (Week 2 Task 03)
`php/praktik.php`, `php/data.php`	Data dari server sendiri, bukan cURL ke Supabase; Week 7: `praktik.php` menjadi front controller (21 baris)
`php/Model/`, `php/View/`, `php/Controller/` *Week 7*	`Practice`, `Profession`, `Schedule` pindah dari `php/classes/`; `PracticeRepository` → `PracticeModel`; HTML → `practice-page.php`; alur → `PracticeController`
`router.php`, `.htaccess`, `.gitignore`, `README.md`, CI	Pengunci folder, `data/` tidak di-commit, cara menjalankan + arsitektur MVC, cek sintaks PHP; `check-refs.mjs` membaca link di `View/` dari folder front controller
Dihapus	`js/supabase.js`, `supabase/` (migrasi SQL), `.mcp.json`, skill Supabase. Semuanya masih ada di riwayat git.

13 Pengujian

Diuji di kode yang sama dengan yang akan didemokan, dengan data awal yang baru disalin dari `app/data/contoh/`.

Skenario	Cara	Hasil
Semua 18 endpoint: baca, tambah, ubah, hapus	curl	✓ bentuk JSON sama dengan Supabase (nama kolom, relasi seperti `medicines {generic_name, unit}`)
Isian salah (tanggal 30 Feb, nama kosong, jumlah 0, jam tutup < buka, nama layanan kembar)	curl	✓ ditolak dengan pesan berbahasa Indonesia
Stok: masuk, keluar melebihi sisa, koreksi, resep FEFO, obat belum diizinkan	curl	✓ sama dengan aturan SQL lama
Booking publik: hari tutup, > 60 hari, nama kosong, booking ke-4 dari nomor yang sama	curl	✓ ditolak; 3 booking pertama masuk
Daftar → onboarding → Beranda; pasien, uang masuk, layanan standar, jadwal, buka praktik, contoh obat, keluar	Chrome (otomatis)	✓ semua toast berhasil, data tersimpan
Konfirmasi booking dari Beranda; selesaikan resep	Chrome	✓ badge turun; stok Vitamin B 60 → 53
Akun lain mengubah pasien Bidan Siti	curl	✓ 404
`/data/users.json`, `/app/…`	curl + Chrome	✓ 404
Semua halaman setelah login; overflow 11 halaman × 3 lebar layar	Chrome	✓ tanpa error JS / request gagal; overflow 0px
Week 7: 100 request API (baca, tambah, ubah, hapus, error 401/404/405/409/422/429) dijalankan ke kode sebelum dan sesudah MVC	skrip Node	✓ semua jawaban (kode status + JSON) sama persis
Week 7: `php/praktik.php` dengan 7 variasi URL (contoh, 2 praktik, tidak ada, kode aneh, `?p[]=`)	curl + diff	✓ HTML dan kode status sama persis
Week 7: alur Chrome diulang di kode MVC (daftar, onboarding, pasien, uang, layanan, jadwal, obat, booking publik, konfirmasi, resep)	Chrome (otomatis)	✓ tanpa error JS; stok Vitamin B 60 → 53
Sintaks PHP & JS, link file lokal	`php -l`, `node --check`, `check-refs.mjs`	✓ semua valid

14 Tanya-jawab & batasan

Pertanyaan	Jawaban
Kenapa HTML-nya tidak diganti PHP?	Tampilan tetap file HTML (Week 2) dan menjadi View di MVC. PHP mengurus data lewat API (Controller + Model). Satu-satunya halaman yang dibuat PHP adalah `php/praktik.php`, tugas Week 5–7.
Apa bedanya `api/*.php` dengan Controller?	`api/bookings.php` hanya pintu masuk: membaca method HTTP dan `?view=`, lalu memanggil method `BookingController`. Proses dan pemanggilan Model ada di class Controller, jadi alamat API untuk JavaScript tidak berubah.
Kenapa disimpan di file JSON, bukan database?	Database belum ada di materi. Semua akses file lewat class `JsonStore`, jadi saat materi database datang cukup class itu yang diganti.
Bagaimana server tahu siapa yang login?	Setelah login, id akun disimpan di `$_SESSION`. Browser hanya menyimpan cookie berisi nomor session acak, yang ikut terkirim di setiap `fetch`.
Bagaimana data antar-praktik tidak tercampur?	Class `Account` hanya membuka folder `data/akun-<id>/` milik akun yang login. Tidak ada parameter URL untuk memilih folder lain.
Kenapa tombol Google/Apple dan "lupa kata sandi" hilang?	Keduanya butuh layanan luar (penyedia login, pengirim email). Hanya disembunyikan lewat `js/config.js`; halamannya masih ada.

Batasan yang diketahui (rencana sprint berikutnya)

- **Belum ada pembatasan percobaan login** (kata sandi bisa dicoba berulang kali).
- **Belum ada token CSRF**; perlindungan saat ini baru cookie `SameSite=Lax` + body JSON.
- **File JSON** cocok untuk satu praktik kecil dan demo; untuk banyak pengguna sebaiknya pindah ke database (materi CRUD).
- Satu akun = satu praktik dengan satu admin; fitur anggota praktik (nakes/staf) dari versi Supabase belum punya halaman pengaturan.
- Pengaturan Nakesa Patient (tampil di aplikasi pasien) tetap tersimpan, tetapi aplikasi pasiennya sendiri belum ada.

Ringkasnya: NAKESA tetap sama di mata pengguna, sementara semua bagian di belakangnya sekarang memakai materi Week 1–7: HTML, JavaScript DOM, fetch API, PHP, OOP, dan susunan MVC.